Engineering Planning with RFCs, Design Documents and ADRs

What are some successful planning approaches engineering teams use as they grow?

GERGELY OROSZ
JUN 21, 2022 • PAID

Q: As our engineering team grows, we feel the need to do more written planning. Which approaches do tech companies use and how do they work?

The question of whether to document engineering planning is an evergreen one. This issue walks through examples of what certain tech companies do and it attempts to showcase some popular approaches. The newsletter closes with advice on how to decide which formats to choose.

Topics:

1. Uber's evolution of planning processes
2. RFCs, Design Documents
3. Reviewing RFCs
4. Architecture Documents
5. Sourcegraph and RFCs
6. Stedi: RFCs and Decision Records
7. Design Docs at Google
8. Examples of RFCs, and companies that use an RFC-like process. See these in a separate article here.

For the rest of this article, I use the term RFC (Request for Comment) to refer to any type of engineering design document, for simplicity.

1. Uber's evolution of planning processes

Uber is a good example of how engineering planning can evolve as a company grows from a few engineers, through to a few hundred, to well over 2,000 software engineers, in less than ten years.

The company has managed to keep an engineering culture in which it can still ship in a reasonably quick and nimble way, even with thousands of engineers. This is key for the company, as the regulatory environment of the gig economy changes frequently, as does the competitive landscape; meaning players in this field need to respond fast, often in a matter of weeks or months. For example, in 2017 Uber shipped tipping functionality in around two months, in response to competitor Lyft launching this feature. This change touched dozens of systems and involved well over twenty engineering teams.

Uber was founded in 2010 and made its first full-time engineering hire in 2011.

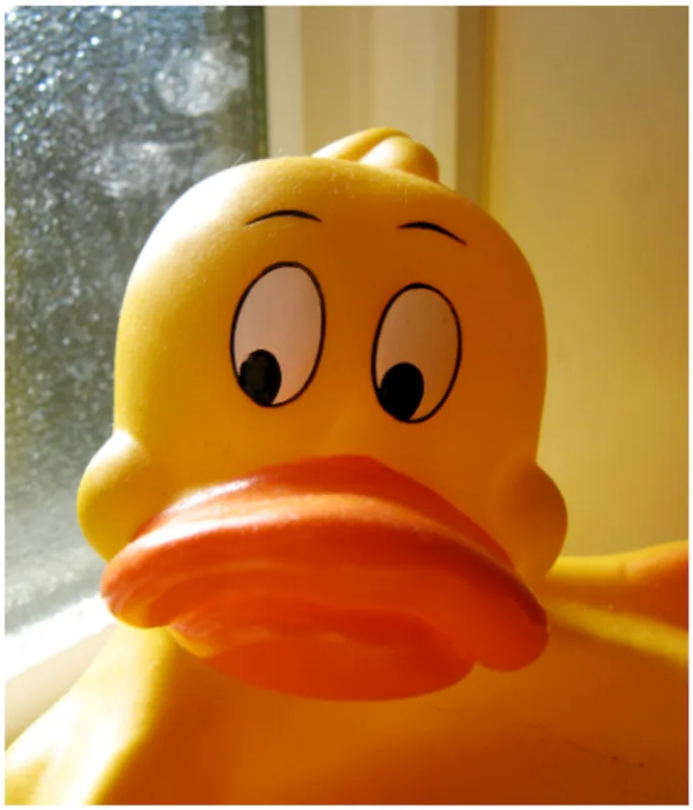
Early on, in around 2012-13**, Uber engineers decided to document new services,** when the company had less than 50 engineers. This decision came from the engineering team, it wasn't mandated from above.

An engineer created and distributed a Google document called DUCK, referring to the "rubber ducking" of their thinking; using simple language to describe proposals, challenges or problems. The approach caught on, and soon the team proposed all new services should have a summary document called a DUCK. In the internal wiki, this page was created:

When starting a new service, it's important to document and vet its architecture. We use a peer review process designed to:

- Catch architectural issues early on.
- Get everyone on the same page with regards to service implementation plans.
- Allow Infra to provision appropriate computing resources.

DUCKs



A DUCK is a service proposal. The acronym doesn't stand for anything. They are kept here.

A DUCK should be written for any new service. It should be created as a Google Doc, and submitted to whomever the author determines they desire feedback from. At a minimum, that distribution list must include the core review group. A DUCK should be submitted *before* major work on the service begins.

No service will be deployed to production without a vetted DUCK.

The first version of DUCK documented, as introduced at Uber around 2013

A few months later, the philosophy of this design document was written down and shared, as this:

DUCK is the new standard format for our service proposal process. The acronym doesn't stand for anything.

When starting a new service, it's important to document and vet its architecture. We use a peer review process designed to:

- *Institute a strong, org-wide practice of information sharing*
- *Provide transparency and good cross-team communication about upcoming systems*
- *Provide historical documentation for our design motivations*
- *Catch potential architectural stumbling blocks before they get to production*
- *Ensure that sufficient thought and time are given to resource allocation before the service needs to go to production*

As context, Uber invested heavily in microservices and teams were encouraged to build their own microservices. By 2016, the company had more than 1,000 microservices. However, with every new service came dependencies; either upon other services, or other services which depended on these new systems.

DUCKs were sent to all engineers subscribed to a mailing list dedicated to DUCKs. Initially, relatively few such documents were sent, and these DUCKs helped teams discover dependencies – often before starting to code. As Uber's engineering team grew to hundreds of engineers, the approach of using DUCKs – designed initially for new services – started to show cracks, with non-backend teams also looking to use this format.

Uber evolved the DUCK format into the RFC, a Request for Comment document. Engineering groups like Backend, Web and Mobile started to create segmented mailing lists to which these documents could be sent, to reduce overall noise as their quantity increased.

The process was also tweaked so that more complex proposals had "approver" fields. This was added to make sure key people read the document and could "sign" if they were happy or add objections if not.

Templates were created to make it easier for engineers to remember key information: for example, service SLAs for services, third party libraries for mobile applications, and

so on.

Here are two example templates with suggestions the company used, at one time:

Services:

- List of approvers
- Abstract (what is the project about?)
- Architecture changes
- Service SLAs
- Service dependencies
- Load & performance testing
- Multi data-center concerns
- Security considerations
- Testing & rollout
- Metrics & monitoring
- Customer support considerations

Mobile:

- Abstract (what is the project about?)
- UI & UX
- Architecture changes
- Network interactions detailed
- Library dependencies
- Security concerns
- Testing & rollout
- Analytics
- Customer support considerations
- Accessibility

As teams grew and learned from failures, some groups added extra check points to avoid past mistakes. For example, when changing the functionality of Payments, regional and legal considerations needed to be checked. When touching systems storing data, GDPR needed to be considered after the regulation was introduced.

A common question was when should an RFC be written? My take on the question was this:

- For small changes: don't bother. Just make the change.
- For changes that are non-trivial and have dependencies: consider writing one.
- The effort to write an RFC should be proportionate to the complexity of the task. If the work is moderately complex, you should get done with the RFC quickly, and many sections might not apply. For complex projects with many dependencies and many risks, you'll have to spend more time on this.

As Uber grew to well over 2,000 engineers, the RFC process started to cause friction. The biggest problems were these:

- **Noise**. By this time, hundreds of RFCs went out weekly, most of them to large mailing lists, and engineers also sent them directly to each other. While this was great for visibility, it overwhelmed more experienced engineers who were asked to look at many RFCs.
- **Ambiguity on which work needed an RFC.** Every team had autonomy to decide how and when to write an RFC. At hundreds of teams, this meant many different interpretations.
- **Discoverability**. RFCs were stored as Google Docs, and finding these documents was not easy. Some teams stored them in Google Drive folders, others linked to them in wiki pages.

Led by one of Uber's principal engineers, the company overhauled this process and rolled out a tiered Engineering Planning Process. The changes were these:

- **Tooling**. Uber built a tool where all engineering planning documents and Product Requirements Documents (PRDs) were stored. This tool was searchable, with

approvers clearly marked, and approval tasks integrated with Uber's task systems, Phabircator and JIRA.

- **Simplifying for smaller changes, more formal for larger changes.** The company introduced lightweight templates for changes limited to team scopes, to help with the problem of overly long documents for relatively simple changes. More heavyweight templates were created for large-scale changes with organization or company-wide impacts.
- **A tiered approach**. The company defined the process to decide how "critical" a change was. For the most critical changes, formal reviews were put in place where experienced engineers in an organization would do reviews on a weekly basis, on top of the asynchronous reviews. For less critical changes, teams had the freedom to adopt the approach they wanted; they could do one of these reviews but were encouraged to keep working as they did so.

2. RFCs and Design Documents

Many Big Tech companies and high-growth startups have ended up with a process similar to Uber's, and settle on a form of the Design Document or RFC process.

Here's how it usually works:

Product Requirement Documents (PRDs) are commonly run side by side with an engineering design document. This is because if product managers don't specify what they'd like the team to build, it's likely the company doesn't have a writing culture.

See a collection of PRD templates collected by Lenny Rachitsky, the author of Lenny's Newsletter - the most popular publication for product managers.

RFCs or Design Docs are written by engineers for non-trivial projects, before they start *meaningful* work. There are usually no hard rules around this. These documents are written to share context, the suggested approach, tradeoffs, and to invite feedback.

The goal of writing and sending out such a document is to decrease the time taken to complete a project, by getting key feedback early. It's down to the engineers working

on the project to decide how an RFC fits into their workflow. Here are a few different approaches I've seen, depending on the project:

- **Prototype, then RFC, then build**. This is typical for projects with lots of unknowns – like building on a new framework – which can be quickly answered with prototyping. The RFC shares a more complete plan, perhaps with some gaps. The team gets feedback, then builds.
- **RFC, wait for feedback, then build.** For projects that have lots of dependencies – or many other teams are dependent on what's being built – it might result in faster progress overall to wait for all the feedback from other teams.
- **RFC, then start building while awaiting feedback.** For projects with some complexity, but when the team is pretty certain there won't be many questions, it's pragmatic to start building right as they send out the RFC. The team can easily incorporate feedback, and even if they need to make changes, not much time is wasted.
- **Build, then write an RFC.** In my observation, this pattern is fairly common. Engineers build what they need to. Then they decide to write an RFC, retroactively documenting the decisions made. This might be because the project got more complex than planned, or to comply with RFC "rules." It's an awkward situation, as the team rarely wants to get feedback, they just do a tick box exercise. But it can be useful to have a document for historic reasons, and getting feedback from other teams on issues at this point is still better than stumbling across the same issue, after the team has moved on to another project.

3. Reviewing RFCs

RFCs are written to solicit feedback on design decisions. So, how do you get this feedback? There are a few different ways, and each has its tradeoffs.

Asynchronous feedback via comments on the document is a common approach many companies use. Uber and Google gather feedback this way on most RFCs shared as Google Docs. GitHub does the same, but using GitHub's comment functionality.

The benefits of asynchronous feedback are there's no need to schedule, it can be done anytime and it works well across time-zones and locations.

The drawbacks of this approach are:

- Tactical feedback which focuses on small, often unimportant details, is more common to receive than feedback on the "big picture."
- Long back-and-forth comment threads are not uncommon and this format is not suited for lengthy discussions.
- Feedback can be often shallow.
- It's hard to tell if someone really read the document and if so, in how much detail.

Synchronous meetings to discuss an RFC is another option. The biggest downside is that this approach needs a lot more time commitment from everyone. To this day, Amazon follows this format in giving feedback to its mandatory 6-pager plans, as I cover in the article Inside Amazon's Engineering Culture.

The benefit of this approach is that people are usually more engaged, the feedback is often at a higher-level, and it isn't 'bogged down' in granular detail. It's a lot easier to sort out differences of opinion on the spot that might easily turn into a never-ending comment thread in an asynchronous format.

The drawback is this could be a waste of many people's time. They might show up with nothing to add, or there might be fundamental objections that mean a new approach is needed, which will then require another, similar meeting. It's also hard to fit several such reviews into a week, especially across time-zones and locations.

A hybrid approach tends to become popular at larger organizations. In this approach, the document is sent out for review asynchronously, ahead of time. A meeting is called if the complexity of the project warrants it, or if there's too many comments piling up. People with relevant input to offer via comments are invited. Others are welcome if they have input, but they're encouraged to give it ahead of time.

My view is the format you choose is contextual. You need a different approach for a project impacting twenty teams where all have some feedback, versus when you're

building a service used only by your team and one partner team.

My suggestion is to never forget the goal of this exercise; it's to reduce the time to ship the project, by getting feedback early. Ask yourself, which approach will help save most time?

4. Architecture Documents

RFCs and design documents are a great way to get higher-level feedback on your approach, *before* starting the work. But what about once you've decided the approach; should you start coding and let the code be the source of truth?

In some cases, there is value in creating an artifact to describe your reasoning, in the form of an architecture document. Twitter staff engineer Ken Lee explains the difference between this and an RFC:

> "I usually recommend design docs (RFCs or whatever you want to call them) to give everyone a bird's eye view of requirements and general architecture of the project, so that folks can comment on missing or erroneous requirements/architecture before it's started.
>
> "Then document a lot in the repo (living docs) as you build whatever it is...always document design decisions (aka ADRs) and unusual steps needed to set it up or develop in it. A.k.a., document as if you'll be hit by a bus. Your coworkers will appreciate it as well as your eventual forgetful self."

An architecture document is written to document your decisions, and less for getting feedback on these decisions. It's down to the project, the team and the engineering culture to decide when to write an architecture document. At FinTech startup Mollie – a competitor to Stripe – it is common to create an architecture document after getting feedback on the RFC.

There are a few popular formats for architecture documents, and each company tends to have a favorite:

- **ADRs** (Architecture Design Records) are probably the most popular format due to being created for use with Git, as Markdown files, and their simple structure. See an overview of ADRs and another summary.
- **arc42** is another format popular with some teams. Read more about this approach and see an example arc42 document through a chess engine.
- **The C4 model** is a more involved, diagramming software architecture, defining four diagram levels for use: context, containers, components and code. It was created by independent consultant and author Simon Brown. Read more about the C4 methodology.

5. Sourcegraph and RFCs

Sourcegraph is a code search scaleup with close to 200 software engineers. The company has been using RFCs heavily, and makes several of them available publicly. They publish an RFC page with their templates and links to public RFCs.

The company has four different templates, depending on the nature of the proposal. I found this distinction useful and suggest that other companies categorize the nature of their documents and how they differ, like this. Here's a taste of RFC structures at Sourcegraph:

1. Framing problems, proposing solutions and making decisions:

- Summary
- Background
- Problem
- Proposal
- Definition of success

2. Surfacing a tension.

- My tension
- My proposal

- What happens if we do nothing?

I really like how the company conceived of this category. In my experience, tensions are common and they're almost always dealt with verbally, or over email. This template suggests making it personal; to share the personal tension and how it affects the person being impacted in their day-to-day work.

3. Adding data to pings.

- Summary
- Background
- Problem
- Proposal

This is an example of a sufficiently common case that Sourcegraph decided it was worth creating a template for.

4. Proposing a process or handbook change.

- Summary
- Background
- Problem
- Proposal
- Definition of success

This template has more detailed context for handbook changes, including pointers to include pull requests of the change.

How has the RFC process worked for Sourcegraph? To answer this question, I reached out to CTO Beyang Liu, who shared the following:

> "RFCs have generally served us well as a lightweight way to solicit feedback from stakeholders across the organization. Having an artifact for the discussion, and a source of truth that evolves with the discussion is super valuable. You could, in theory, use an issue tracker rather than a Google doc. However, issue trackers are

> more like long threads, whereas an RFC enables the author to update the text as discussion evolves.
>
> "As we've grown, we've also hit some friction points with RFCs. In particular, they've become more formal over time, partly because we've standardized the formatting - like with metadata calling out approvers and stakeholders and target dates in the header. In another part, because we haven't sufficiently defined what they are for newer folks. I'd say newer folks, especially non-devs, read more into their formality to the point where some people view it as more of an "official decision process," which wasn't their original intent."

Asked how the RFCs process might change at Sourcegraph, Beyang said this:

> "If I were to venture a guess as to how things will evolve moving forward, I would say this: RFCs are likely to remain an important planning tool that enables lightweight but efficient communication. However, we may adopt a different process for formal decision making. This is an active area of discussion we're currently having."

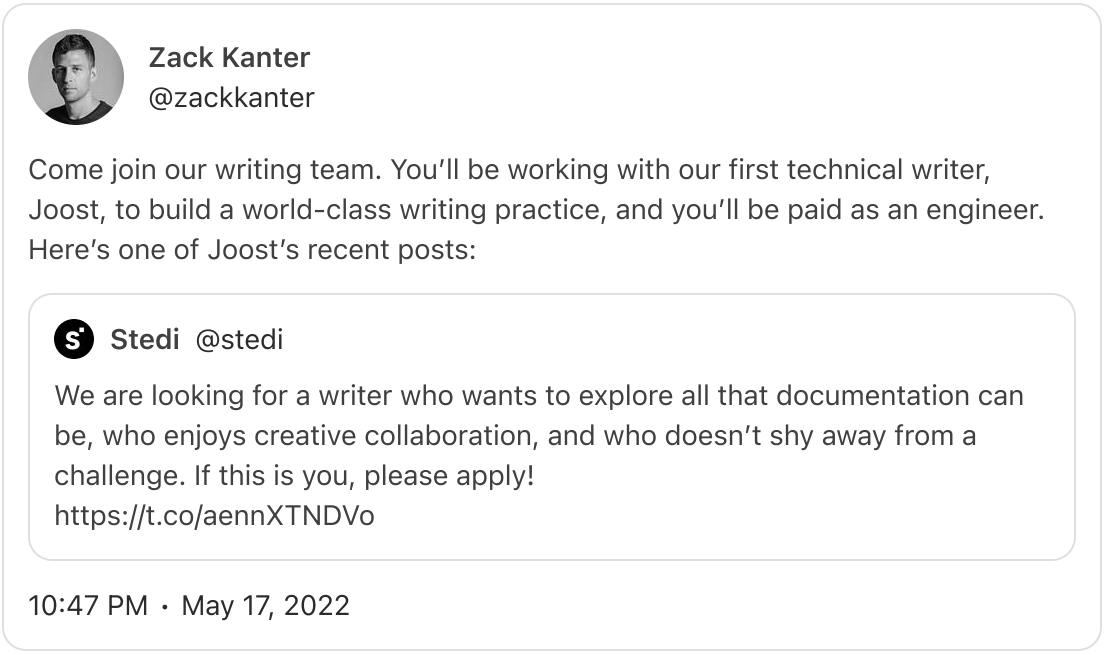
6. Stedi: RFCs and Decision Records

Stedi is a logistics scaleup which has raised $71 million over several funding rounds. The company provides a cloud platform for EDI integrations. EDI is the electronic interchange format that companies use to exchange information with each other. Read more about what is EDI and why it's important.

In something unique to the company, 6 years after founding about 90% of all employees are software engineers. Of the around 85 employees, about 75 are software engineers, around 5 are designers, and the company has hired its first product manager.

One of the reasons the company has successfully grown to this size, has to do with a very deliberate writing culture put in place by the founder CEO, Zack Kanter. It's testament to how well this has worked, given the company ran effectively with one product manager – their CEO playing this role – and scaled decision making. The

company is now building out an internal writing team, paying technical writers just like software engineers:



So how do Stedi people discuss engineering decisions internally? It should be no surprise that they use RFC-like formats a lot. They also have more rules around how they work with an evolution of this format called Decision Records.

RFCs were introduced early at the company. They're written as .MD files on GitHub, and comments are made also using GitHub. RFCs are typically used for broad, complex projects, company-wide technical mandates or API changes.

The company has an RFC template people can use as a starting point and diverge from as needed. The template is this:

Status

Draft | Proposed | Published

Type

Experimental | Informational | Best Current Practice | Required

Scope

The scope of work affected by the RFC document.

Notational conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 RFC2119, RFC8174 when, and only when, they appear in all capitals, as shown here, and without quotes.

Context

A description of the problem space and/or requirements that this RFC is set to address.

Decision

A description of the proposed solution, which sets out the specifications of the implementation of the RFC. Should include specific details on how the solution addresses the requirements as laid out in the Context section.

Consequences

Any consequences that are in play as a result of the decision being put into place.

Decision Records are a more lightweight format that Stedi introduced, shortly after starting to use RFCs. These documents are usually written as Google Docs. The company uses this not just for technical decisions, but for a variety of tasks. For example, they wrote a Decision Record for the introduction of a training platform, and for moving their recruitment process from one vendor to another.

Decision records are used for two key reasons:

1. To codify a decision that's already been made.

2. As a forcing function to get alignment on a decision that needs to be made

Here is a template of a decision record the company uses.

Stedi uses Decision Records as part of the everyday workflow, and has built tooling to make them easier to work with:

- **Decider**. Every decision record has a decider; the person who officially makes the decision. Most decision records are decided by people on the team. Still, someone is always named as the decider who makes this official.
- **Changing a Decision Record**. Once a record is complete, it cannot be edited. However, it can be overturned with a new Decision Record.
- **Transparency**. Decision Records are shared across the whole company. When someone creates a new Decision Record, they add this Google Doc to a shared drive with all other Decision Records. This action triggers a Slack bot to post on a channel dedicated to Decision Records and RFCs.
- **Workflow**. Once a Decision Record is submitted, it will eventually end up as either Accepted, Rejected or Withdrawn, following a discussion.

The most important function the company sees in writing, is to use it as a tool for thinking. This means the purpose of writing important things down is not to record actions taken. Rather, it implies writing should come before action, particularly in the domain of software development. Stedi is huge on this principle.

The writing culture at Stedi is tightly coupled with how complex the EDI domain it operates in is; and how, for the most part, there have been no dedicated product managers at the company. By writing before implementing, the company has managed to build the right things, and to surface questions and issues quickly.

I'll close by quoting an internal memo at Stedi that explains why they place so much emphasis on thinking before coding, and therefore write before they code (emphasis mine):

> "Writing a doc is not a perfunctory gesture, and asking for a doc on something is not a punishment or a mechanism for control. Writing a doc is a way of i) surfacing

> requirements and assumptions, ii) driving clarity of reasoning stemming from those requirements and assumptions. **Without this step, our software has little hope of delivering the results we want over the long term."**
>
> As a final thought, not everyone has to write docs here. Some people just want to execute, and there is plenty of room for that, too – but if you just want to execute, you'll be executing on the plan, architecture, or implementation described in someone else's doc. Our domain is too complex and our ambitions too large to build software willy-nilly."

7. Design Docs at Google

Former Google principal engineer Malte Ubl wrote an article on how Google uses Design Docs, which are similar to what other companies call RFCs.

Malte summarizes the goals for Design Docs at Google:

- **Catch design issues early.** The earlier these are caught, the cheaper they are to fix.
- **Consensus**. Reach consensus across the organization about a design.
- **Cross-cutting concerns**. Consider cross-organization and cross-cutting concerns.
- **Information sharing.** Scale knowledge, especially from senior engineers to the rest of the organization.
- **Organizational memory.** Design Docs form the basis of this.
- **Historic artifact.** A summary artifact which can be referred to, later.

Design documents are informal, meaning there is no strict or uniform format. In the article, Malte summarized a structure worth considering:

- Context and scope
- Goals and non-goals
- The design

    - System-context diagram
    - APIs
    - Data storage
    - Code and pseudo-code
    - Degree of constraint
- Alternatives considered
- Cross-cutting concerns

When not to write a design document is always a great question. Here's how Malte sees when it's not needed:

> "A clear indicator that a doc might not be necessary are design docs that are really implementation manuals. If a doc basically says "This is how we are going to implement it" without going into trade-offs, alternatives, and explaining decision making (or if the solution is so obvious as to mean there were no trade-offs), then it would probably have been a better idea to write the actual program right away.
>
> "Finally, the overhead of creating and reviewing a design doc may not be compatible with prototyping and rapid iteration. However, most software projects do have a set of actually known problems. Subscribing to agile methodologies is not an excuse for not taking the time to get solutions to actually known problems right. Additionally, prototyping itself may be part of the design doc creation. "I tried it out and it works" is one of the best arguments for choosing a design."

Read more on how Google uses Design Documents.

8. Examples of RFCs, and companies that use an RFC-like process

Hundreds of companies use RFC-like processes, and an increasing number of startups adopt such a setup from the start.

See a list of some of these companies, and see more RFC templates here.

Takeaways

I've observed both software engineers and engineering leaders struggle with the topic of RFCs and engineering design documents. Software engineers are often unsure if it's worth writing down the plan for an architecture, or if it's a waste of time. Engineering leaders are often uncertain if they should enforce the writing down of things, or if this creates bureaucracy and more red tape.

My view is it's helpful for every engineer and every organization to come to an approach that works for them in their given context – and then to iterate on it. That said, here are a few things you may consider as takeaways from this article.

As an engineer, if you want to get feedback before you implement, there is no more scalable way to do this, than by writing a short document. You could just whiteboard your idea, or explain it to the person next to you, but then this idea will be gone as soon as the whiteboard is cleared, or it won't spread beyond the other person. To scale an idea – meaning others at different times can understand it – you need to write it down.

In the article Becoming a better writer as a software engineer, I argue writing is an increasingly important skill for software engineers:

> "With remote work and distributed teams becoming more common, strong writing skills are a baseline for most engineering leadership positions. In fact, writing is becoming so important that companies like Amazon start their engineering manager screening process with a writing exercise, as I covered in the issue Hiring Engineering Managers. (...)
>
> "If you are a software engineer at a high-growth startup or Big Tech, the chances are you write more words per day than you write code."

As an engineering leader, if your team doesn't have a culture of writing things down, you will struggle far more to scale the organization, than if you have it in place. I've gotten several reach outs from CTOs and VPs of Engineering who all found it hard to grow their teams to above 100-200 engineers, when there was no culture of sharing decisions in written form.

These organizations have all relied on some of their longest-tenured engineers playing an architect role, by overseeing decisions and giving feedback on them. However, at some point, they could no longer keep up; it became clear to these leaders they needed a more asynchronous, more distributed feedback process, in which these long-tenured engineers are not the bottlenecks.

Many high-growth tech companies end up with similar models in how to get feedback on engineering work, before the work is started. Uber, Google, Sourcegraph and Stedi all set out to solve their own problems, and did not copy their engineering process from other places. And yet, they all ended up with similar processes.

Uber – a famously "not invented here" organization – re-invented the necessity of writing engineering plans down because it worked much better than the alternatives. I am not saying you should copy the way Uber operated in its early days. But I would invite you to consider how is it that – completely independently from each other and while not *wanting* to copy any practices elsewhere – Uber introduced DUCKs when it was still small, Amazon instituted a culture of writing from day one and Google came up with Design Docs?

Consider the similarities between why Uber decided to introduce the DUCK format around 2013, and how Malte Ubl summarized why Google uses Design Documents:

Uber (around 2013)	Google (around 2022)
• **Catch issues early.** Catch potential architectural stumbling blocks before they get to production. • **Information sharing**. Institute a strong, org-wide practice of information sharing. • **Cross-cutting concerns** and transparency**.** Provide transparency and good cross-team communication about upcoming systems. • **Historic artifact.** Provide historical documentation for our design motivations • **Resource allocation.** Ensure sufficient thought and time are given to resource allocation before the service needs to go to production.	• **Catch design issues early.** The earlier these are caught, the cheaper they are to fix. • **Information sharing.** Scale knowledge, especially from senior engineers to the rest of the org. • **Cross-cutting concerns**. Consider cross-organization and cross-cutting concerns. • **Historic artifact.** A summary artifact that can be referred to, later. • **Consensus.** Reach consensus across the organization about a design. • **Organizational memory.** Design Docs form the basis of this.

pragmaticengineer.com

Similarities between why Uber and Google uses RFCs / Design Docs

Facebook is a notable standout with how little of an RFC culture it has, but can you afford to follow them? Out of all Big Tech, Facebook places the least emphasis on documentation, as I reveal in Inside Facebook's Engineering Culture: Part 2.

In this article, I suggest caution if you decide to follow this approach. Facebook makes its organization work with little documentation, counter-balancing this omission with world-class monitoring, alerting and automated rollout systems across several environments, hiring very good engineers and those engineers staying for an unusually long time. As I write in that article:

> "Facebook has less documentation and testing in-place than most tech companies. This approach works for a few reasons: historic ones, engineers having long tenures, Facebook hires some of the best engineers, they built systems to counter the lack of documents and testing. I would not suggest deliberately following this approach, especially not in the age of asynchronous and remote work."

Engineers who write tests for their code, often ship more maintainable code. Engineers who write design docs for their architecture, often ship more maintainable architecture. Writing a design document and writing tests for the code have similar benefits.

When an engineer writes tests for their code, they often discover edge cases otherwise missed, and the code they propose to merge is already of higher quality. Asking for a code review brings more feedback, and improves the quality and maintainability of that code.

The same is true for designing software. Writing down your thoughts forces you to make them more concrete, with less ambiguity. This means people reading the document will understand and be able to interpret your thoughts better than if you shared your ideas on a whiteboard or explained it verbally. An easier to understand artifact will then bring higher quality, more relevant feedback.

We should acknowledge several software engineers will push back against writing design documents or RFCs. I would suggest understanding the underlying reason why. Is it because they find writing too time-consuming? If so, what makes it time-consuming? Is it the lack of practice they have had in writing, or that their thoughts are less focused than they believe?

Writing is increasingly important for software engineers, and a culture of writing down engineering decisions is important for engineering teams, in order to scale beyond a certain point. RFCs and design documents are a step on this journey. I would suggest experimenting with one of the many formats we've covered in this issue if you have not yet done so.

Don't forget the real goal of RFCs. It's not to document something, or to follow a process. The goal is to ship software that works as expected, *faster*. If you can build software faster that works as expected for everyone – for you, stakeholders, and customers – without needing to use a tool like an RFC, then don't use an RFC! In some cases, you might be better off with prototyping, or in some rarer cases with shipping straight to production.

Practice the RFC approach to master this tool, and as you do so, you'll learn about the tradeoffs of this approach.

An RFC – like unit tests, code reviews, feature flags, canarying and other engineering approaches – is a tool that can help ship better software, faster. Take inspiration from this article to either try it out, or to iterate on how you write RFCs.

Thanks to Beyang, Marek, Sid, Zack for their contributions to this newsletter.

How did you like this issue? 🤔

Amazing • Great • Good • OK • So-so

101 Likes · 1 Restack

13 Comments

Write a comment...

Simarpreet Aug 1, 2022 ❤ Liked by Gergely Orosz

Overall, it was a good read, but one thing I couldn't help but notice that "building consensus" doesn't seem like a prevalent reason for RFC design docs. In this issue, it is only mentioned under Google's process.

I, however, acknowledge that it could be happening implicitly through feedback and comments on RFCs, but still It was quite surprising for me, as IMO it is should be key motivations to write RFC's specially for products/orgs with a lot engineers and teams.

♡ LIKE (1) 💬 REPLY •••

1 reply by Gergely Orosz

Kevin Coulombe Jun 28, 2022 ❤ Liked by Gergely Orosz

Source Graph mentions a more formal decision process than RFCs. You also make a distinction between RFCs and ADRs as separate artifacts.

I was thinking that for more critical decisions, the RFC would evolve into an ADR to document the consensus in more details. But I don't see the value of keeping both.

Do I just lack imagination on what merits a formal decision process? I'd love to hear more on your views into how this fits a larger collaboration strategy...

LIKE (1) REPLY •••

3 replies by Gergely Orosz and others

11 more comments...